RETINEX-BASED IMAGE ENHANCEMENT FOR LOW-LIGHT CONDITIONS

Digital Image Processing (DIP) Project Report

Research Focus: Study of Illumination-Reflectance Decomposition

Submitted by: M.Neha Choudhary

Enrollment Number:24CS003290

Class: CSE “G”

Submitted To: Mr. Manish Tiwari

Department: Computer Science and Engineering

Institution: Sir Padampat Singhania University (SPSU)

ABSTRACT

Low-light images often suffer from poor visibility, reduced contrast and difficulty in identifying important scene details. This project investigates Retinex-based image enhancement with a specific focus on illumination-reflectance decomposition.

The project is implemented in Python using Jupyter Notebook, OpenCV, NumPy and Matplotlib. A simplified Retinex approach estimates illumination through Gaussian smoothing and derives an approximate reflectance component. The illumination is enhanced using a square-root transformation, after which the image is reconstructed. The method is evaluated on three images and compared with conventional gamma correction.

Mean brightness, grayscale contrast and entropy are calculated to describe changes in the processed images. The experiments show that both enhancement methods increase brightness across the tested images, while their effects on contrast vary. The findings illustrate the potential and limitations of simplified Retinex-based enhancement.

Keywords: Digital Image Processing, Retinex, Low-Light Enhancement, Illumination-Reflectance Decomposition, Gaussian Filtering, Gamma Correction.

RESEARCH FOCUS

STUDY OF ILLUMINATION-REFLECTANCE DECOMPOSITION

1.Background and Development of Retinex Theory

1.1 Retinex Theory

Retinex theory is associated with the work of Edwin H. Land and John J. McCann on lightness and colour perception. It provides a foundation for understanding how an image's appearance is affected by illumination and object reflectance.

A simplified image formation model is:

$$I(x,y) = R(x,y) \times L(x,y)$$

where I is the observed image, R is reflectance, and L is illumination. This model motivates methods that estimate lighting and use the estimate to improve the visibility of dark regions.

1.2 Single-Scale Retinex

Jobson, Rahman and Woodell studied a centre/surround Retinex method in 1997. Their work examined the relationship between image rendition and dynamic-range compression. Single-scale Retinex uses a surrounding region to estimate broad illumination variations and enhance image appearance.

This research provides a foundation for practical Retinex image-enhancement techniques. The Gaussian smoothing used in the present project is a simplified illumination-estimation approach inspired by this general idea.

1.3 Multi-Scale Retinex

Jobson, Rahman and Woodell subsequently presented Multi-Scale Retinex, which combines information from multiple spatial scales. Using multiple scales helps balance local detail enhancement, colour consistency and dynamic-range compression.

Compared with a single-scale approach, multi-scale processing can capture illumination variations at different spatial ranges. However, its performance depends on the chosen scales and parameters.

1.4 Retinex-Based Low-Light Enhancement

Later research expanded Retinex methods to address low-light enhancement, noise, colour distortion and uneven illumination. Some methods explicitly estimate illumination and reflectance, while others combine Retinex ideas with optimization or deep learning.

A review by the authors of A Survey on Image Enhancement for Low-Light Images discusses traditional approaches, including intensity transformations, histogram equalization and Retinex-based methods, alongside learning-based techniques.

These studies demonstrate that improving brightness is only one aspect of low-light enhancement. Preserving details, colours and natural appearance is also important.

1.5 Comparison with Gamma Correction

Gamma correction is a point-processing method that transforms pixel intensities using a power function. For normalized intensity I and gamma value γ, it can be expressed as:

$I_{enhanced} = I^{\gamma}$

When γ is less than 1, darker intensity values are increased. Gamma correction is computationally simple, but it does not explicitly estimate illumination and reflectance.

In this project, gamma correction serves as a baseline against which the simplified Retinex implementation is compared.

2. Proposed Retinex-Based Enhancement Method

2.1 Tools and Technologies Used

The following tools and technologies were used for implementation:

- Python: Programming language used to implement the image-enhancement algorithm.
- Jupyter Notebook: Environment used to write, execute, and test the code.
- OpenCV: Used for image loading, colour-space conversion, Gaussian blurring, and image processing.
- NumPy: Used for numerical calculations and array operations.
- Matplotlib: Used to display and compare the original and enhanced images.
- Pandas: Used to organize and save experimental results in CSV files.

2.2 Proposed System Architecture

The system follows a sequence of image-processing operations to enhance low-light images.

System workflow:

Input Low-Light Image

↓

Image Loading and Preprocessing

↓

Illumination Estimation Using Gaussian Blur

↓

Illumination–Reflectance Decomposition

↓

Illumination Enhancement

↓

Image Reconstruction

↓

Save Enhanced Image

↓

Calculate Brightness, Contrast, and Entropy

↓

Compare with Gamma Correction

↓

Analyze Results

2.3 Image Acquisition and Preprocessing

The first step is to load a low-light image from the computer using OpenCV. The image is converted from BGR format to RGB format for correct colour display in Matplotlib. The pixel values are normalized to the range 0 to 1 to simplify mathematical operations.

Three images were used to test the proposed method:

1. The Stars.jpg
2. Nightly LA.jpg
3. download (5).jpg

Using multiple images helps examine the behaviour of the enhancement method under different image conditions.

2.4 Illumination–Reflectance Decomposition

Retinex theory represents an observed image as a combination of illumination and reflectance. The image can be expressed approximately as:

$$R(x,y)=I(x,y)/L(x,y)$$

where:

- $I(x,y)$ represents the observed image.
- $R(x,y)$ represents the reflectance component.
- $L(x,y)$ represents the illumination component.
- (x,y) represent the pixel coordinates.

Reflectance contains information about the objects and their surface characteristics, while illumination represents the lighting conditions.

In this project, a Gaussian blur is applied to the grayscale image to estimate the illumination component. The estimated illumination is restricted to a minimum value to reduce division by very small numbers.

The reflectance component is estimated using:

$$R(x,y)=\frac{I(x,y)}{L(x,y)}$$

The estimated reflectance is clipped to the range 0 to 1 for the implementation.

2.5 Illumination Enhancement

After decomposition, the illumination component is enhanced using a square-root transformation:

$$L'(x,y)=\sqrt{L(x,y)}$$

This transformation increases the intensity of darker illumination values and helps improve visibility in low-light regions.

The enhanced image is reconstructed by multiplying the estimated reflectance by the enhanced illumination:

$$I'(x,y)=R(x,y) \times L'(x,y)$$

The reconstructed image is clipped to the range 0 to 1 and converted into an appropriate image representation for display and saving.

This implementation is a simplified educational Retinex-based method. It demonstrates illumination–reflectance decomposition and enhancement rather than implementing every feature of advanced Retinex algorithms.

2.6 Gamma Correction for Comparison

Gamma correction is used as a baseline method to compare its results with the proposed Retinex-based method. For normalized image intensity, gamma correction can be expressed as:

I'(x,y)=I(x,y)^\gamma

In this experiment, the gamma value is set to 0.6. Since the value is less than 1, darker pixel intensities are increased, making the image appear brighter.

Both methods are applied to the test images, and their output images and numerical measurements are compared.

2.7 Performance Evaluation Metrics

Three metrics are used to evaluate the original and enhanced images.

2.7..1 Mean Brightness

Mean brightness represents the average intensity of the image. An increase in mean brightness indicates that the image has become brighter.

2.7.2 Contrast

Contrast is measured using the standard deviation of grayscale pixel intensities. A larger value indicates greater variation in pixel intensities. However, higher contrast does not always mean better visual quality.

2.7.3 Entropy

Image entropy measures the distribution and diversity of grayscale intensity values. It is calculated from the grayscale histogram. Higher entropy can indicate a more varied intensity distribution, but it does not necessarily mean that the image quality is better.

These metrics are used to compare the original image, gamma-corrected image, and Retinex-enhanced image.

2.8 Implementation and Result Storage

The algorithm was implemented and tested in Jupyter Notebook. The enhanced images and intermediate illumination and reflectance components were saved in the results folder. Comparison figures were generated to visually examine the output, while the numerical measurements were stored in CSV files.

The saved outputs support the experimental analysis presented in section 3

.

3.Experimental Results And Discussion

3.1 Experimental Setup

The proposed Retinex-based image enhancement method was implemented using Python in Jupyter Notebook. OpenCV, NumPy, Pandas and Matplotlib were used for image processing, numerical calculations, evaluation and visualization.

Three images were tested: The Stars.jpg, Nightly LA.jpg and download (5).jpg. The results of the proposed method were compared with the original images and a gamma-correction baseline.

3.2 Evaluation Metrics

Three image characteristics were measured:

- Mean Brightness: Represents the average grayscale intensity of an image.
- Contrast: Represents the standard deviation of grayscale intensity values.
- Entropy: Describes the distribution of grayscale intensity values.

These metrics help describe changes in image appearance but do not independently establish overall visual quality.

3.3 Results for The Stars.jpg

METHOD	BRIGHTNESS	CONTRAST	ENTROPY
ORIGINAL	35.945	26.377	6.455
GAMMA CORRECTION	72.705	33.168	6.994
RETINEX	73.952	32.082	7.005

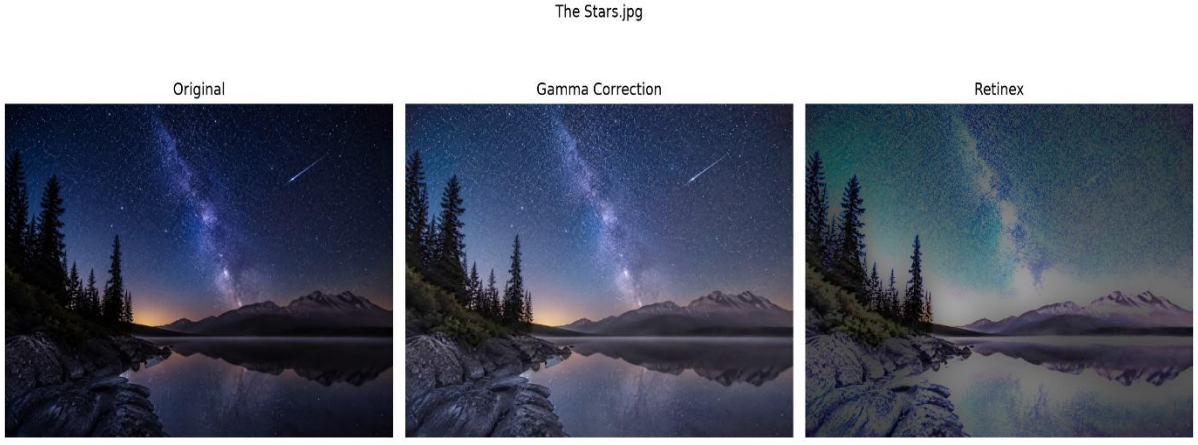


Figure 3.1: Comparison of Original Image, Gamma Correction, and Retinex-Based Enhancement for The Stars.jpg

The Retinex result has a higher mean brightness than the gamma-corrected result. Its contrast is slightly lower, while its entropy is slightly higher. These measurements indicate changes in intensity distribution and should be considered alongside visual inspection.

3.4 Results for Nightly LA.jpg

METHOD	BRIGHTNESS	CONTRAST	ENTROPY
ORIGINAL	29.971	35.207	5.906
GAMMA CORRECTION	62.940	38.280	6.452
RETINEX	64.068	27.643	6.460

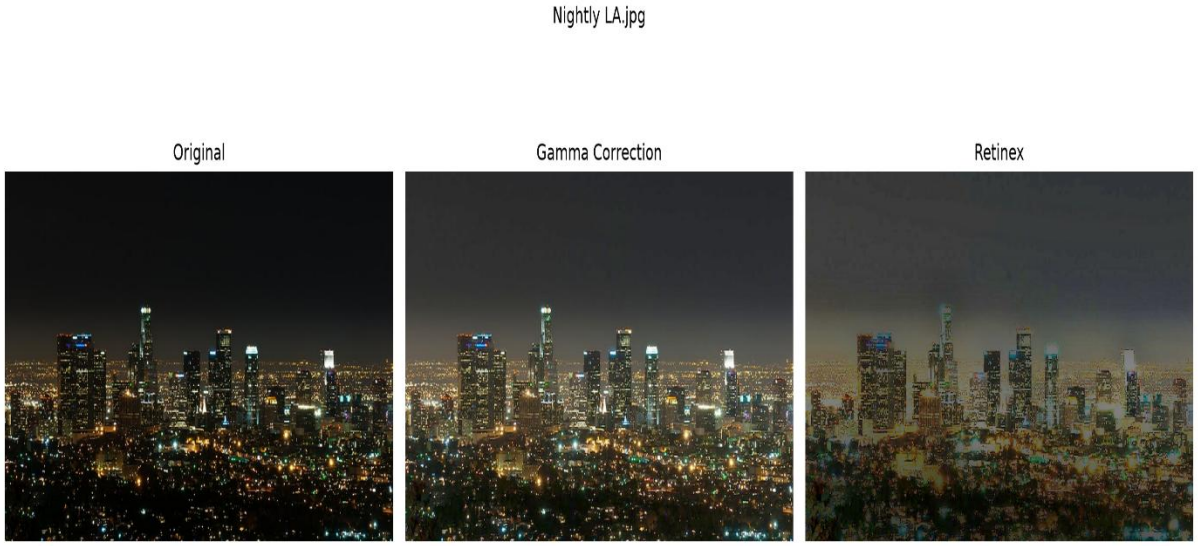


Figure 3.2: Comparison of Original Image, Gamma Correction, and Retinex-Based Enhancement for Nightly LA.jpg

Retinex produced a higher mean brightness than gamma correction for this image. However, its contrast was lower. This suggests that brightness improvement does not necessarily preserve the same intensity variation.

3.5 Results for download (5).jpg

METHOD	BRIGHTNESS	CONTRAST	ENTROPY
ORIGINAL	27.471	32.321	6.102
GAMMA CORRECTION	55.790	43.835	6.630
RETINEX	53.994	40.627	6.633

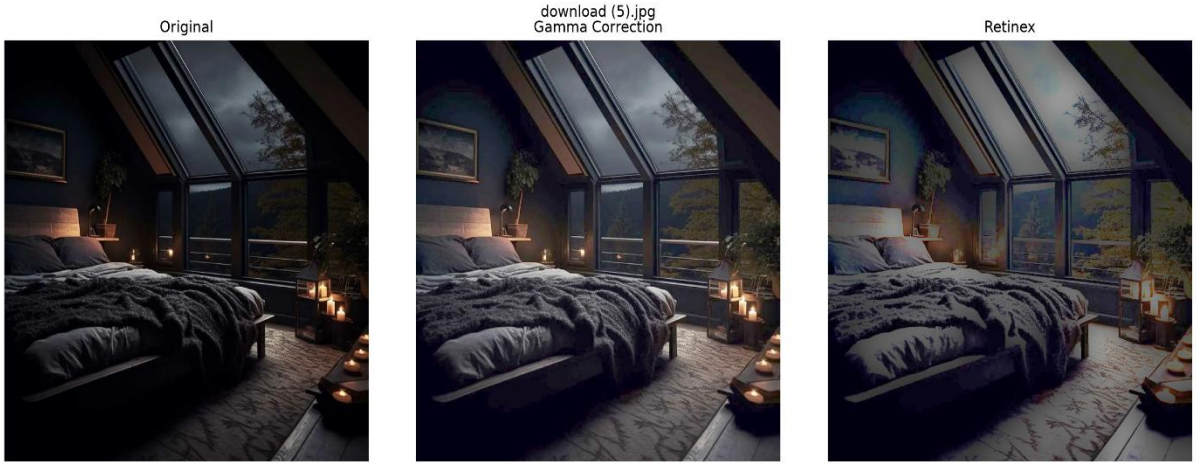


Figure 3.3: Comparison of Original Image, Gamma Correction, and Retinex-Based Enhancement for download (5).jpg

For this image, gamma correction produced higher brightness and contrast than Retinex. Retinex produced slightly higher entropy. Therefore, the performance of the methods varies across images.

3.6 Overall Discussion

The experiments show that both gamma correction and the implemented Retinex method increase brightness for the three tested images. Retinex produces a higher brightness value than gamma correction for two images, but not for the third.

The contrast results vary between images. Entropy is slightly higher for Retinex than gamma correction in all three cases, although higher entropy does not necessarily mean better image quality.

The findings suggest that enhancement performance depends on the characteristics of the input image. Further testing with additional images and more robust quality measures would be useful.

4.Research Findings And Conclusion

Conclusion

This project presents a Retinex-based image enhancement method for improving the visibility of images captured under low-light conditions. The method is based on illumination–reflectance decomposition, in which the observed image is separated into illumination and reflectance components.

Python, OpenCV, NumPy, Matplotlib, and Pandas were used to implement and evaluate the proposed method. Gaussian blur was used to estimate illumination, and a square-root transformation was applied to enhance it. The enhanced image was reconstructed using the reflectance and enhanced illumination components.

The proposed method was tested on three low-light images and compared with gamma correction using a gamma value of 0.6. Both

methods increased the mean brightness of all three images compared with their original versions. The Retinex method produced slightly higher entropy values than gamma correction for all three images. However, gamma correction produced higher contrast values in these experiments.

The results show that the Retinex-based method can improve the brightness of low-light images. However, the results also demonstrate that image enhancement performance depends on the input image and the evaluation metric. Entropy and contrast alone cannot determine the overall visual quality of an enhanced image.

Overall, the project demonstrates the basic principle of Retinex-based image enhancement and the importance of illumination estimation in processing low-light images.

Limitations

The project has the following limitations:

1. The illumination component is estimated using Gaussian blur, which is a simplified approach.

2. Only three images were used for experimental evaluation.

3. The method may produce different results for images with varying lighting conditions and colour distributions.

4. Evaluation was based on brightness, contrast, and entropy, without using reference images or advanced perceptual quality metrics.

5. The implementation does not include advanced techniques such as multi-scale Retinex or deep-learning-based enhancement.

Final Remarks

This project provides a practical introduction to Retinex theory and illumination–reflectance decomposition. The implementation and experimental comparison help explain how low-light image enhancement works and how its performance can be evaluated. Further improvements could make the method more effective for practical image-processing applications.

REFERENCES

1. Land, E. H., and McCann, J. J. (1971). Lightness and Retinex Theory. Journal of the Optical Society of America, 61(1), 1–11.

2. Jobson, D. J., Rahman, Z., and Woodell, G. A. (1997). Properties and Performance of a Center/Surround Retinex. IEEE Transactions on Image Processing, 6(3), 451–462. https://doi.org/10.1109/83.557356

3. Jobson, D. J., Rahman, Z., and Woodell, G. A. (1997). A Multiscale Retinex for Bridging the Gap Between Color Images and the Human Observation of Scenes. IEEE Transactions on Image Processing, 6(7), 965–976. https://doi.org/10.1109/83.597272

4. A Survey on Image Enhancement for Low-light Images. PMC. https://pmc.ncbi.nlm.nih.gov/articles/PMC10070385/

5. OpenCV Documentation. Image Processing. https://docs.opencv.org/

6. NumPy Documentation. https://numpy.org/doc/

7. Matplotlib Documentation. https://matplotlib.org/stable/

8. Pandas Documentation. https://pandas.pydata.org/docs/

APPENDIX A

PYTHON SOURCE CODE

A.1 Library Imports and Project Folder Setup

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
project_dir = Path.cwd()
(project_dir / "input_images").mkdir(exist_ok=True)
(project_dir / "results").mkdir(exist_ok=True)
(project_dir / "screenshots").mkdir(exist_ok=True)
print("OpenCV version:", cv2.__version__)
print("Project folder:", project_dir)
print("Project folders created successfully!")
```

A.2 Loading and Displaying the Original Image.

```
import cv2
import matplotlib.pyplot as plt
img_path = r"C:\Users\Neha\Downloads\The Stars.jpg"
img = cv2.imread(img_path)
if img is None:
    print("Image not found. Please check the
path.")
```

```
else:
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(8, 5))
    plt.imshow(img_rgb)
    plt.title("Original Image - The Stars")
    plt.axis("off")
    plt.show()
    print("Image loaded successfully!")
    print("Image dimensions:", img.shape)
```

A.3 Retinex-Based Image Enhancement and Illumination–Reflectance Decomposition

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
original = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
I = original.astype(np.float32) / 255.0
gray = cv2.cvtColor(original, cv2.COLOR_RGB2GRAY)
illumination = cv2.GaussianBlur(
    gray.astype(np.float32) / 255.0,
    (0, 0),
    sigmaX=25
)
illumination = np.maximum(illumination, 0.05)
```

```
reflectance = I / illumination[:, :, np.newaxis]
reflectance = np.clip(reflectance, 0, 1)
enhanced_illumination = np.sqrt(illumination)
enhanced_illumination =
np.clip(enhanced_illumination, 0, 1)
enhanced = reflectance * enhanced_illumination[:,
:, np.newaxis]
enhanced = np.clip(enhanced, 0, 1)
plt.figure(figsize=(15, 8))
plt.subplot(2, 2, 1)
plt.imshow(original)
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(illumination, cmap="gray")
plt.title("Estimated Illumination")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(np.mean(reflectance, axis=2),
cmap="gray")
plt.title("Estimated Reflectance")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(enhanced)
```

```
plt.title("Retinex Enhanced Image")
plt.axis("off")
plt.tight_layout()
plt.show()
from pathlib import Path
results_dir = Path("results")
results_dir.mkdir(exist_ok=True)
cv2.imwrite(
    str(results_dir / "retinex_enhanced.png"),
    cv2.cvtColor((enhanced * 255).astype(np.uint8),
                 cv2.COLOR_RGB2BGR))
cv2.imwrite(
    str(results_dir / "illumination.png"),
    (illumination * 255).astype(np.uint8)
)
cv2.imwrite(
    str(results_dir / "reflectance.png"),
    (np.mean(reflectance, axis=2) *
255).astype(np.uint8)
)
print("Retinex decomposition completed.")
print("Results saved in:", results_dir.resolve())
```

A.4 Evaluation Metrics: Brightness, Contrast, and Entropy

```
import numpy as np
import cv2
import pandas as pd
from pathlib import Path
def calculate_metrics(image):
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    brightness = np.mean(gray)
    contrast = np.std(gray)
    hist = cv2.calcHist([gray], [0], None, [256],
[0, 256])
    probabilities = hist.ravel() / hist.sum()
    probabilities = probabilities[probabilities >
0]
    entropy = -np.sum(probabilities *
np.log2(probabilities))
    return brightness, contrast, entropy
original_uint8 = original.astype(np.uint8)
enhanced_uint8 = (enhanced * 255).astype(np.uint8)
original_metrics =
calculate_metrics(original_uint8)
enhanced_metrics =
calculate_metrics(enhanced_uint8)
results_table = pd.DataFrame({
```

```
    "Metric": ["Mean Brightness (0-255)",
               "Contrast (Standard Deviation)",
               "Grayscale Entropy (bits)"],
    "Original Image": original_metrics,
    "Enhanced Image": enhanced_metrics
})
print(results_table.to_string(index=False))
Path("results").mkdir(exist_ok=True)
results_table.to_csv("results/evaluation_metrics.csv", index=False)
print("\nEvaluation table saved successfully.")
```

A.5 Multi-Image Retinex Experiment and Performance Comparison

```
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
image_paths = [
    r"C:\Users\Neha\Downloads\The Stars.jpg",
    r"C:\Users\Neha\Downloads\Nightly LA.jpg",
    r"C:\Users\Neha\Downloads\download (5).jpg"
]
```

```
results_dir = Path("results")
results_dir.mkdir(exist_ok=True)
def retinex_enhance(image):
    original_rgb = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB)
    I = original_rgb.astype(np.float32) / 255.0
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    illumination = cv2.GaussianBlur(
        gray.astype(np.float32) / 255.0,
        (0, 0), sigmaX=25)
    illumination = np.maximum(illumination, 0.05)
    reflectance = I / illumination[:, :,
np.newaxis]
    reflectance = np.clip(reflectance, 0, 1)
    enhanced_illumination = np.sqrt(illumination)
    enhanced = reflectance *
enhanced_illumination[:, :, np.newaxis]
    enhanced = np.clip(enhanced, 0, 1)
    return original_rgb, illumination, reflectance,
enhanced
def calculate_metrics(rgb_image):
    gray = cv2.cvtColor(rgb_image,
cv2.COLOR_RGB2GRAY)
    brightness = float(np.mean(gray))
    contrast = float(np.std(gray))
```

```
    hist = cv2.calcHist([gray], [0], None, [256],
[0, 256]).ravel()
    p = hist / hist.sum()
    p = p[p > 0]
    entropy = float(-np.sum(p * np.log2(p)))
    return brightness, contrast, entropy
rows = []
for index, path in enumerate(image_paths, start=1):
    image = cv2.imread(path)
    if image is None:
        print("Could not load:", path)
        continue
    original_rgb, illum, refl, enhanced =
retinex_enhance(image)
    enhanced_u8 = (enhanced * 255).astype(np.uint8)
    output_path = results_dir /
f"enhanced_image_{index}.png"
    cv2.imwrite(
        str(output_path),
        cv2.cvtColor(enhanced_u8,
cv2.COLOR_RGB2BGR)  )
    original_metrics =
calculate_metrics(original_rgb)
    enhanced_metrics =
calculate_metrics(enhanced_u8)
```

```
        rows.append({
            "Image": Path(path).name,
            "Original Brightness": original_metrics[0],
            "Enhanced Brightness": enhanced_metrics[0],
            "Original Contrast": original_metrics[1],
            "Enhanced Contrast": enhanced_metrics[1],
            "Original Entropy": original_metrics[2],
            "Enhanced Entropy": enhanced_metrics[2]
        })
        plt.figure(figsize=(10, 4))
        plt.subplot(1, 2, 1)
        plt.imshow(original_rgb)
        plt.title("Original")
        plt.axis("off")
        plt.subplot(1, 2, 2)
        plt.imshow(enhanced_u8)
        plt.title("Retinex Enhanced")
        plt.axis("off")
        plt.suptitle(Path(path).name)
        plt.tight_layout()
        plt.show()
    if rows:
        comparison = pd.DataFrame(rows)
```

```
    display(comparison.round(3))
    comparison.to_csv(
        results_dir / "multi_image_comparison.csv",
        index=False
    )
    print("Experiment completed. Results saved
to:", results_dir.resolve())
else:
    print("No images were processed. Check the file
paths.")
```

A.6 Comparison of Gamma Correction and Retinex Enhancement

```
from pathlib import Path
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
comparison_rows = []
results_dir = Path("results")
results_dir.mkdir(exist_ok=True)
for path in image_paths:
    bgr = cv2.imread(path)
    if bgr is None:
```

```
        print("Skipping unreadable image:", path)
        continue
    original_rgb = cv2.cvtColor(bgr,
cv2.COLOR_BGR2RGB)
    # Gamma correction
    gamma = 0.6
    gamma_img = np.power(
        original_rgb.astype(np.float32) / 255.0,
        gamma
    )
    gamma_img = np.clip(gamma_img * 255, 0,
255).astype(np.uint8)
    # Retinex enhancement
    _, illumination, reflectance, retinex_float =
retinex_enhance(bgr)
    retinex_img = np.clip(
        retinex_float * 255, 0, 255
    ).astype(np.uint8)
    # Display comparison
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    axes[0].imshow(original_rgb)
    axes[0].set_title("Original")
    axes[1].imshow(gamma_img)
    axes[1].set_title("Gamma Correction")
```

```
    axes[2].imshow(retinex_img)
    axes[2].set_title("Retinex")
    for ax in axes:
        ax.axis("off")
    fig.suptitle(Path(path).name)
    plt.tight_layout()
    plt.savefig(
        results_dir /
f"{Path(path).stem}_method_comparison.png",
        dpi=200,
        bbox_inches="tight"
    )
    plt.show()
    # Calculate performance metrics
    for method, result in [
        ("Original", original_rgb),
        ("Gamma Correction", gamma_img),
        ("Retinex", retinex_img)
    ]:
        brightness, contrast, entropy =
calculate_metrics(result)
        comparison_rows.append({
            "Image": Path(path).name,
```

```
            "Method": method,
            "Brightness": brightness,
            "Contrast": contrast,
            "Entropy": entropy
        })
# Save comparison results
comparison_df = pd.DataFrame(comparison_rows)
display(comparison_df.round(3))
comparison_df.to_csv(
    results_dir / "method_comparison_metrics.csv",
    index=False
)
print("Comparison figures and metrics saved
successfully.")
```